EXERCÍCIO 2: LARGE LANGUAGE MODELS

Maria Inês Matos PG61533
Patrícia Bastos PG60287

1. e 2. Modelos escolhidos

codellama-7b-instruct.Q2_K

```
[codecarbon INFO @ 15:58:09] >>> Tracker's metadata:
[codecarbon INFO @ 15:58:09]   Platform system: macOS-26.3.1-arm64-arm-64bit-Mach-O
[codecarbon INFO @ 15:58:09]   Python version: 3.14.4
[codecarbon INFO @ 15:58:09]   CodeCarbon version: 3.2.6
[codecarbon INFO @ 15:58:09]   Available RAM : 16.000 GB
[codecarbon INFO @ 15:58:09]   CPU count: 8 thread(s) in 1 physical CPU(s)
[codecarbon INFO @ 15:58:09]   CPU model: Apple M2
[codecarbon INFO @ 15:58:09]   GPU count: 1
[codecarbon INFO @ 15:58:09]   GPU model: Apple M2
[codecarbon INFO @ 15:58:11] Emissions data (if any) will be saved to file /Users/patriciabastos/Documents/universidad
llama_context: n_ctx_seq (2048) < n_ctx_train (16384) -- the full capacity of the model will not be utilized
Question: What is the capital of Portugal?
Output: Lisbon
[codecarbon INFO @ 15:58:15]
Graceful stopping: collecting and writing information.
Please wait a few seconds...
[codecarbon INFO @ 15:58:15] Energy consumed for RAM : 0.000004 kWh. RAM Power : 3.0 W
[codecarbon INFO @ 15:58:17] Energy consumed for all CPUs : 0.000002 kWh. Total CPU Power : 1.5821 W
[codecarbon INFO @ 15:58:18] Energy consumed for all AppleSilicon GPUs : 0.000000 kWh. Total GPU Power : 0.0521 W
[codecarbon INFO @ 15:58:18] 0.000006 kWh of electricity and 0.000000 L of water were used since the beginning.
[codecarbon INFO @ 15:58:18] Done!
```

Meta-Llama-3-8B-Instruct-Q2_k

```
[codecarbon INFO @ 16:05:15] >>> Tracker's metadata:
[codecarbon INFO @ 16:05:15]   Platform system: macOS-26.3.1-arm64-arm-64bit-Mach-O
[codecarbon INFO @ 16:05:15]   Python version: 3.14.4
[codecarbon INFO @ 16:05:15]   CodeCarbon version: 3.2.6
[codecarbon INFO @ 16:05:15]   Available RAM : 16.000 GB
[codecarbon INFO @ 16:05:15]   CPU count: 8 thread(s) in 1 physical CPU(s)
[codecarbon INFO @ 16:05:15]   CPU model: Apple M2
[codecarbon INFO @ 16:05:15]   GPU count: 1
[codecarbon INFO @ 16:05:15]   GPU model: Apple M2
[codecarbon INFO @ 16:05:18] Emissions data (if any) will be saved to file /Users/patriciabastos/Documents/universidade
llama_context: n_ctx_seq (2048) < n_ctx_train (8192) -- the full capacity of the model will not be utilized
Question: What is the capital of Portugal?
Output: Lisbon.
[codecarbon INFO @ 16:05:23]
Graceful stopping: collecting and writing information.
Please wait a few seconds...
[codecarbon INFO @ 16:05:23] Energy consumed for RAM : 0.000004 kWh. RAM Power : 3.0 W
[codecarbon INFO @ 16:05:24] Energy consumed for all CPUs : 0.000002 kWh. Total CPU Power : 1.5567 W
[codecarbon INFO @ 16:05:26] Energy consumed for all AppleSilicon GPUs : 0.000000 kWh. Total GPU Power : 0.182 W
[codecarbon INFO @ 16:05:26] 0.000006 kWh of electricity and 0.000000 L of water were used since the beginning.
[codecarbon INFO @ 16:05:26] Done!
```

3. Tasks escolhidas

Escolhemos as seguintes 20 tasks:

- 11
- 12
- 14
- 16
- 17
- 18
- 19
- 20
- 56
- 57
- 58
- 59
- 61
- 62
- 63
- 64
- 65
- 66
- 67
- 68

4.

Para o **3-shot** prompting escolhemos as tasks **2**, **3**, e **4**.
Para a medição da energia consumida e do tempo de inferencia para cada geração usamos o **CodeCarbon**.

Para as medições recorremos ao uso de um script de forma a serem automatizadas.

5. BLEU score

Recorremos à utilização de um script para automatizar o cálculo do “BLEU score”, através do **CodeBLEU**, mais especificamente “Pip compatible CodeBLEU metric implementation available for linux/macos/win”.
Abaixo tem um exemplo de utilização, conforme as instruções que se encontram no **repositório do github do codeBLEU**[1]

```
result = calc_codebleu([ref], [hyp], lang="python", weights=(0.25, 0.25, 0.25, 0.25), tokenizer=None)
```

Onde **ref** é o código de referência disponibilizado pelo dataset mbpp e hyp é a code completition feita pela llm
À direita encontra-se um resumo do score para cada task.

```
==================================================
SUMMARY
==================================================

codellama:
  task_11:
    output_0: CodeBLEU=0.1036, N-gram=0.0045, Syntax=0.0208, Dataflow=0.3824
    output_3: CodeBLEU=0.0510, N-gram=0.0000, Syntax=0.0208, Dataflow=0.1765
  task_12:
    output_0: CodeBLEU=0.1872, N-gram=0.0204, Syntax=0.4000, Dataflow=0.2857
    output_3: CodeBLEU=0.1510, N-gram=0.0341, Syntax=0.1000, Dataflow=0.4286
  task_14:
    output_0: CodeBLEU=0.3354, N-gram=0.0112, Syntax=0.3000, Dataflow=1.0000
    output_3: CodeBLEU=0.3365, N-gram=0.0166, Syntax=0.3000, Dataflow=1.0000
  task_16:
    output_0: CodeBLEU=0.2174, N-gram=0.0354, Syntax=0.2222, Dataflow=0.5714
    output_3: CodeBLEU=0.2989, N-gram=0.1028, Syntax=0.5556, Dataflow=0.4286
  task_17:
    output_0: CodeBLEU=0.1876, N-gram=0.0251, Syntax=0.2500, Dataflow=0.4000
    output_3: CodeBLEU=0.1934, N-gram=0.0485, Syntax=0.2500, Dataflow=0.4000
  task_18:
    output_0: CodeBLEU=0.1092, N-gram=0.0043, Syntax=0.1786, Dataflow=0.2500
    output_3: CodeBLEU=0.0399, N-gram=0.0002, Syntax=0.0714, Dataflow=0.0833
  task_19:
    output_0: CodeBLEU=0.2675, N-gram=0.0326, Syntax=0.5000, Dataflow=0.5000
    output_3: CodeBLEU=0.2675, N-gram=0.0326, Syntax=0.5000, Dataflow=0.5000
  task_20:
    output_0: CodeBLEU=0.0671, N-gram=0.0002, Syntax=0.0270, Dataflow=0.2353
    output_3: CodeBLEU=0.1390, N-gram=0.0323, Syntax=0.1892, Dataflow=0.2941
  task_56:
    output_0: CodeBLEU=0.1477, N-gram=0.0133, Syntax=0.2667, Dataflow=0.2778
    output_3: CodeBLEU=0.1413, N-gram=0.0209, Syntax=0.3000, Dataflow=0.2222
  task_57:
    output_0: CodeBLEU=0.2331, N-gram=0.0689, Syntax=0.2308, Dataflow=0.5556
    output_3: CodeBLEU=0.1025, N-gram=0.0010, Syntax=0.2308, Dataflow=0.1667
  task_58:
    output_0: CodeBLEU=0.2913, N-gram=0.0114, Syntax=0.1111, Dataflow=1.0000
    output_3: CodeBLEU=0.3691, N-gram=0.0906, Syntax=0.2222, Dataflow=1.0000
  task_59:
    output_0: CodeBLEU=0.2615, N-gram=0.0799, Syntax=0.1111, Dataflow=0.7500
    output_3: CodeBLEU=0.3000, N-gram=0.0402, Syntax=0.1111, Dataflow=1.0000
  task_61:
    output_0: CodeBLEU=0.1636, N-gram=0.0799, Syntax=0.1633, Dataflow=0.3143
    output_3: CodeBLEU=0.1492, N-gram=0.0819, Syntax=0.1837, Dataflow=0.2286
  task_62:
    output_0: CodeBLEU=0.5510, N-gram=0.0955, Syntax=1.0000, Dataflow=1.0000
    output_3: CodeBLEU=0.5510, N-gram=0.0955, Syntax=1.0000, Dataflow=1.0000
  task_63:
    output_0: CodeBLEU=0.1544, N-gram=0.0130, Syntax=0.1667, Dataflow=0.4167
    output_3: CodeBLEU=0.1204, N-gram=0.0163, Syntax=0.1111, Dataflow=0.3333
  task_64:
    output_0: CodeBLEU=0.3269, N-gram=0.0567, Syntax=0.3846, Dataflow=0.8000
    output_3: CodeBLEU=0.3906, N-gram=0.0082, Syntax=0.5385, Dataflow=1.0000
  task_65:
    output_0: CodeBLEU=0.1768, N-gram=0.0089, Syntax=0.1852, Dataflow=0.5000
    output_3: CodeBLEU=0.1484, N-gram=0.0068, Syntax=0.0741, Dataflow=0.5000
  task_66:
    output_0: CodeBLEU=0.5423, N-gram=0.0779, Syntax=1.0000, Dataflow=1.0000
    output_3: CodeBLEU=0.5423, N-gram=0.0779, Syntax=1.0000, Dataflow=1.0000
  task_67:
    output_0: CodeBLEU=0.0910, N-gram=0.0028, Syntax=0.2642, Dataflow=0.0833
    output_3: CodeBLEU=0.0502, N-gram=0.0179, Syntax=0.0189, Dataflow=0.1250
  task_68:
    output_0: CodeBLEU=0.1769, N-gram=0.0100, Syntax=0.1613, Dataflow=0.5238
    output_3: CodeBLEU=0.6364, N-gram=0.3443, Syntax=0.8387, Dataflow=1.0000
```

```
==================================================
SUMMARY
==================================================

meta-llama:
  task_11:
    output_0: CodeBLEU=0.1209, N-gram=0.0024, Syntax=0.0625, Dataflow=0.4118
    output_3: CodeBLEU=0.0490, N-gram=0.0005, Syntax=0.0417, Dataflow=0.1471
  task_12:
    output_0: CodeBLEU=0.1790, N-gram=0.0021, Syntax=0.1000, Dataflow=0.5714
    output_3: CodeBLEU=0.5368, N-gram=0.0699, Syntax=1.0000, Dataflow=1.0000
  task_14:
    output_0: CodeBLEU=0.3082, N-gram=0.0022, Syntax=0.2000, Dataflow=1.0000
    output_3: CodeBLEU=0.3809, N-gram=0.1060, Syntax=0.3000, Dataflow=1.0000
  task_16:
    output_0: CodeBLEU=0.2801, N-gram=0.0223, Syntax=0.3333, Dataflow=0.7143
    output_3: CodeBLEU=0.1791, N-gram=0.0158, Syntax=0.1111, Dataflow=0.5714
  task_17:
    output_0: CodeBLEU=0.2510, N-gram=0.0019, Syntax=0.1250, Dataflow=0.8000
    output_3: CodeBLEU=0.1622, N-gram=0.0485, Syntax=0.1250, Dataflow=0.4000
  task_18:
    output_0: CodeBLEU=0.1167, N-gram=0.0022, Syntax=0.0595, Dataflow=0.4000
    output_3: CodeBLEU=0.1038, N-gram=0.0007, Syntax=0.3095, Dataflow=0.1000
  task_19:
    output_0: CodeBLEU=0.3222, N-gram=0.0016, Syntax=0.5000, Dataflow=0.7500
    output_3: CodeBLEU=0.3930, N-gram=0.0163, Syntax=0.6429, Dataflow=0.8750
  task_20:
    output_0: CodeBLEU=0.2188, N-gram=0.0073, Syntax=0.4324, Dataflow=0.4118
    output_3: CodeBLEU=0.0689, N-gram=0.0055, Syntax=0.0270, Dataflow=0.2353
  task_56:
    output_0: CodeBLEU=0.1581, N-gram=0.0042, Syntax=0.3333, Dataflow=0.2778
    output_3: CodeBLEU=0.1313, N-gram=0.0149, Syntax=0.2667, Dataflow=0.2222
  task_57:
    output_0: CodeBLEU=0.1820, N-gram=0.0067, Syntax=0.1538, Dataflow=0.5556
    output_3: CodeBLEU=0.1065, N-gram=0.0039, Syntax=0.0769, Dataflow=0.3333
  task_58:
    output_0: CodeBLEU=0.2885, N-gram=0.0014, Syntax=0.1111, Dataflow=1.0000
    output_3: CodeBLEU=0.3023, N-gram=0.0193, Syntax=0.1111, Dataflow=1.0000
  task_59:
    output_0: CodeBLEU=0.3072, N-gram=0.0088, Syntax=0.1111, Dataflow=1.0000
    output_3: CodeBLEU=0.3555, N-gram=0.0402, Syntax=0.3333, Dataflow=1.0000
  task_61:
    output_0: CodeBLEU=0.1526, N-gram=0.0313, Syntax=0.1633, Dataflow=0.3143
    output_3: CodeBLEU=0.1758, N-gram=0.0894, Syntax=0.2245, Dataflow=0.2857
  task_62:
    output_0: CodeBLEU=0.4920, N-gram=0.0022, Syntax=0.8571, Dataflow=1.0000
    output_3: CodeBLEU=0.5510, N-gram=0.0955, Syntax=1.0000, Dataflow=1.0000
  task_63:
    output_0: CodeBLEU=0.1239, N-gram=0.0022, Syntax=0.0556, Dataflow=0.4167
    output_3: CodeBLEU=0.1751, N-gram=0.0123, Syntax=0.1667, Dataflow=0.5000
  task_64:
    output_0: CodeBLEU=0.4988, N-gram=0.0069, Syntax=0.9231, Dataflow=1.0000
    output_3: CodeBLEU=0.3057, N-gram=0.0100, Syntax=0.3846, Dataflow=0.8000
  task_65:
    output_0: CodeBLEU=0.2269, N-gram=0.0040, Syntax=0.2222, Dataflow=0.6667
    output_3: CodeBLEU=0.2173, N-gram=0.0041, Syntax=0.1852, Dataflow=0.6667
  task_66:
    output_0: CodeBLEU=0.1544, N-gram=0.0090, Syntax=0.1429, Dataflow=0.4444
    output_3: CodeBLEU=0.6217, N-gram=0.2279, Syntax=1.0000, Dataflow=1.0000
  task_67:
    output_0: CodeBLEU=0.1247, N-gram=0.0039, Syntax=0.1887, Dataflow=0.2917
    output_3: CodeBLEU=0.0790, N-gram=0.0009, Syntax=0.1887, Dataflow=0.1042
  task_68:
    output_0: CodeBLEU=0.2407, N-gram=0.0164, Syntax=0.4839, Dataflow=0.3810
    output_3: CodeBLEU=0.2921, N-gram=0.0342, Syntax=0.4839, Dataflow=0.5714


Results saved to results/codebleu_results.csv
```

[1] https://github.com/k4black/codebleu#usage

5. codeBLEU score

Score por modelo e estratégia de prompting

	CodeLLama	Meta-Llama
	média	
0-shot	0.2296	0.2373
3-shot	0.2489	0.2593

Score por modelo

	CodeLLama	Meta-Llama
média	0.2393	0.2483

O modelo Meta-Llama apresenta um score mais elevado em relação ao CodeLlama, logo tem melhor performance em termos de codeBLEU score.

Score por estratégia de prompting

	0-shot	3-shot
média	0.2335	0.2541

A estratégia 3-shot apresenta um score mais elevado em relação a 0-shot, logo tem melhor performance em termos de codeBLEU score.

Apesar do Meta-Llama e a estratégia 3-shot terem melhor performance, todos os valores de codeBLEU score encontram-se bastante baixos.

5. ngram_match score (BLEU)

Score por modelo e estratégia de prompting

	CodeLLama	Meta-Llama
	média	
0-shot	0.0326	0.0069
3-shot	0.0534	0.0412

Score por modelo

	CodeLLama	Meta-Llama
média	0.0430	0.0241

Score por estratégia de prompting

	0-shot	3-shot
média	0.0198	0.0473

O modelo **CodeLlama** e a estratégia **3-shot** têm melhor performance em termos de **ngram_match score (BLEU)**.

5. weighted_ngram_match score (BLEU-weighted)

Score por modelo e estratégia de prompting

	CodeLLama	Meta-Llama
	média	
0-shot	0.0462	0.0441
3-shot	0.0671	0.0567

Score por modelo

	CodeLLama	Meta-Llama
média	0.0567	0.0504

Score por estratégia de prompting

	0-shot	3-shot
média	0.0452	0.0619

O modelo **CodeLlama** e a estratégia **3-shot** têm melhor performance em termos de **weighted_ngram_match score (BLEU-weighted)**.

5. syntax_match score (AST match)

Score por modelo e estratégia de prompting

	CodeLLama	Meta-Llama
	média	
0-shot	0.2972	0.2779
3-shot	0.3308	0.3489

Score por modelo

	CodeLLama	Meta-Llama
média	0.3140	0.3134

Score por estratégia de prompting

	0-shot	3-shot
média	0.2876	0.3399

O modelo **CodeLlama** e a estratégia **3-shot** têm melhor performance em termos de **syntax_match score (AST match)**.

5. dataflow_match score

Score por modelo e estratégia de prompting

	CodeLLama	Meta-Llama
	média	
0-shot	0.5423	0.6204
3-shot	0.5443	0.5906

Score por modelo

	CodeLLama	Meta-Llama
média	0.5433	0.6055

Score por estratégia de prompting

	0-shot	3-shot
média	0.5813	0.5675

O modelo **Meta-Llama** e a estratégia **0-shot** têm melhor performance em termos de **dataflow_match score**.

5. Conclusão

Os resultados anteriores são calculados sem tokenization. Os valores com a tokenization default não têm alterações significativas.

Em geral, o modelo **CodeLlama** e a estratégia **3-shot** obtiveram melhores resultados em comparação ao modelo Meta-Llama e a estratégia 0-shot, respetivamente. Apesar disso, as diferenças de valor não são muito distantes

Em geral, o BLEU score apenas avalia o quão parecido uma resposta é da referência, considerando apenas "token match" e ignorando a importância de "keywords", precisão sintática e semântica. Isto faz com que o BLEU score não seja adequado para a avaliação de código, logo preferiu-se a utilização do CodeBleu, que tem mais em conta todos esses aspetos em que o BLEU falha.
Mesmo assim, o CodeBleu score continua a não ter em conta se o código que foi gerado para o problema está correto, e sim o quão parecido (em relação ao mencionado anteriormente) é ao código de referência. Isto não avalia com 100% de precisão a qualidade e performance da LLM já que vários problemas podem ter diferentes implementações corretas e igualmente eficientes.

6. pass@1 metric

	CodeLlama		Meta-Llama	
task	0-shot	3-shot	0-shot	3-shot
11	0	1	0	1
12	0	1	0	1
14	0	0	0	0
16	1	0	0	0
17	1	1	0	1
18	0	1	0	1
19	1	1	0	0
20	0	0	0	0
56	0	0	0	0
57	0	1	0	0

	CodeLlama		Meta-Llama	
task	0-shot	3-shot	0-shot	3-shot
58	0	0	0	1
59	0	0	0	0
61	1	0	0	0
62	1	1	0	1
63	0	0	0	0
64	0	0	0	1
65	0	1	0	0
66	1	1	0	1
67	0	0	0	0
68	0	1	0	0
total	6/20 30%	10/20 50%	0/20 0%	8/20 40%

6. pass@1 metric

CodeLlama:

- **0-shot:** obteve 30% de sucesso pois só conseguiu resolver 6 tarefas de um total de 20
- **3-shot:** a adição de exemplos fez com que a taxa de sucesso passasse de 30% para 50%

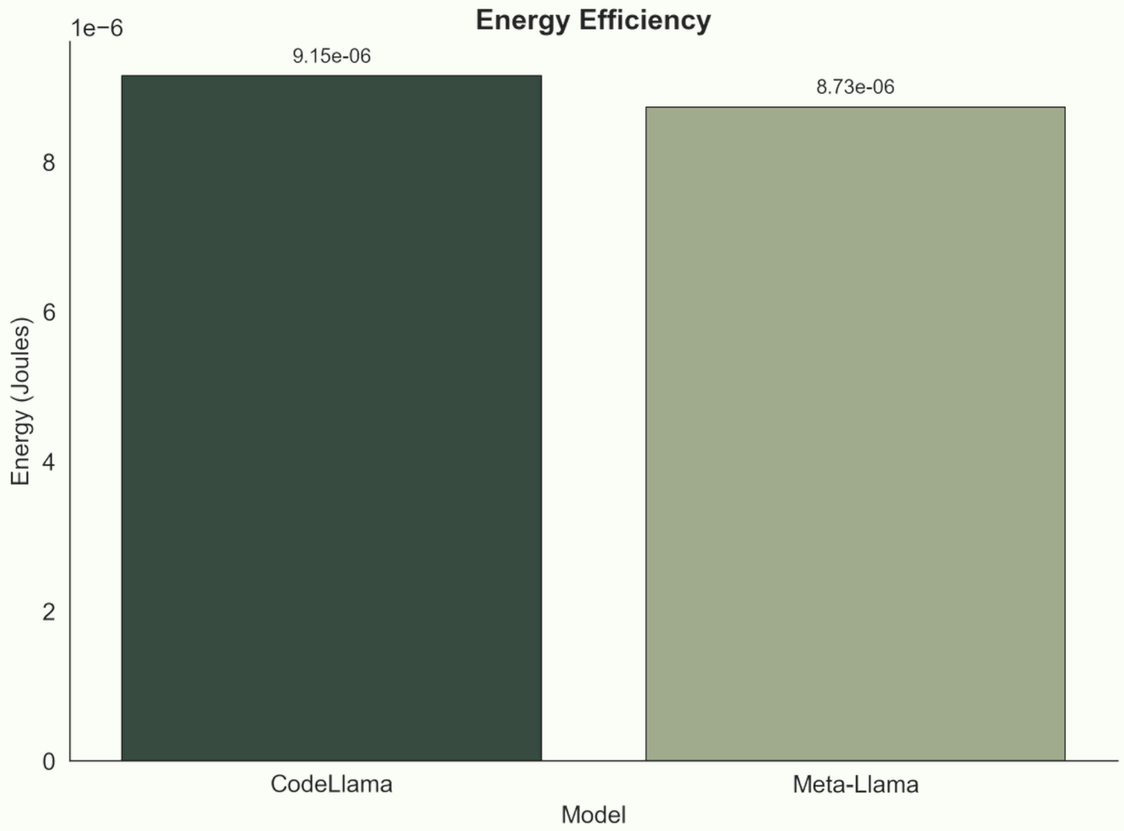
Meta-Llama:

- **0-shot:** sem exemplos, o Meta-Llama não teve sucesso em nenhuma tarefa, ficando assim com uma taxa de sucesso de 0%
- **3-shot:** com a adição de exemplos verificou-se um aumento significativo na taxa de sucesso, passando para 40%

Conclusão: CodeLlama com 3-shot prompting é o melhor em termos de functional correctness com 50% de pass@1. A estratégia 3-shot é claramente superior em ambos os modelos e é fundamental para o Meta-Llama ter alguma utilidade.

7. a)

(a) Which model would you use for local Python code generation if you prioritize energy efficiency, and why?

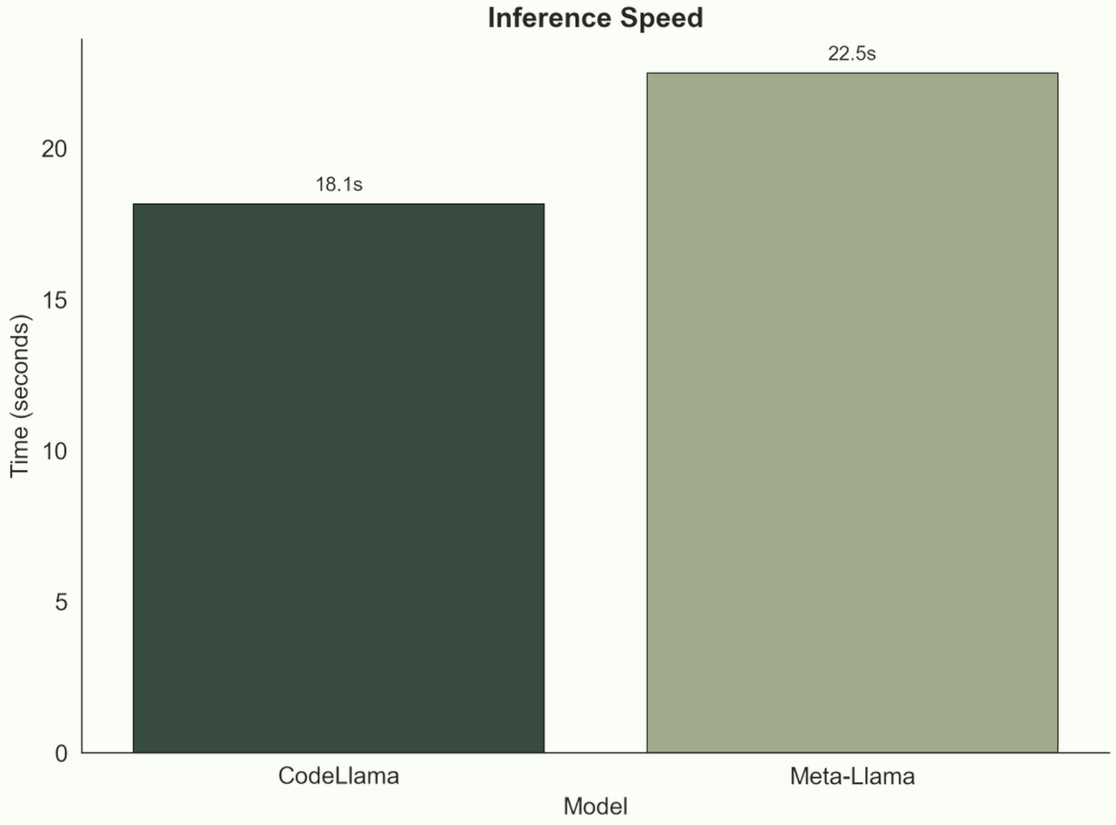


O CodeLlama apresenta um consumo energético médio de 9.15×10^{-6} J, valor muito próximo ao do Meta-Llama (8.73×10^{-6} J). Assim, se o objetivo for priorizar a eficiência energética, o **Meta-Llama** é a melhor escolha entre os dois modelos.

Nota: Para os valores dos gráficos foi usada a média entre os resultados obtidos com a estratégia 0-shot e com a estratégia 3-shot

7. b)

(b) Which model would you use for local Python code generation if you prioritize inference speed, and why?

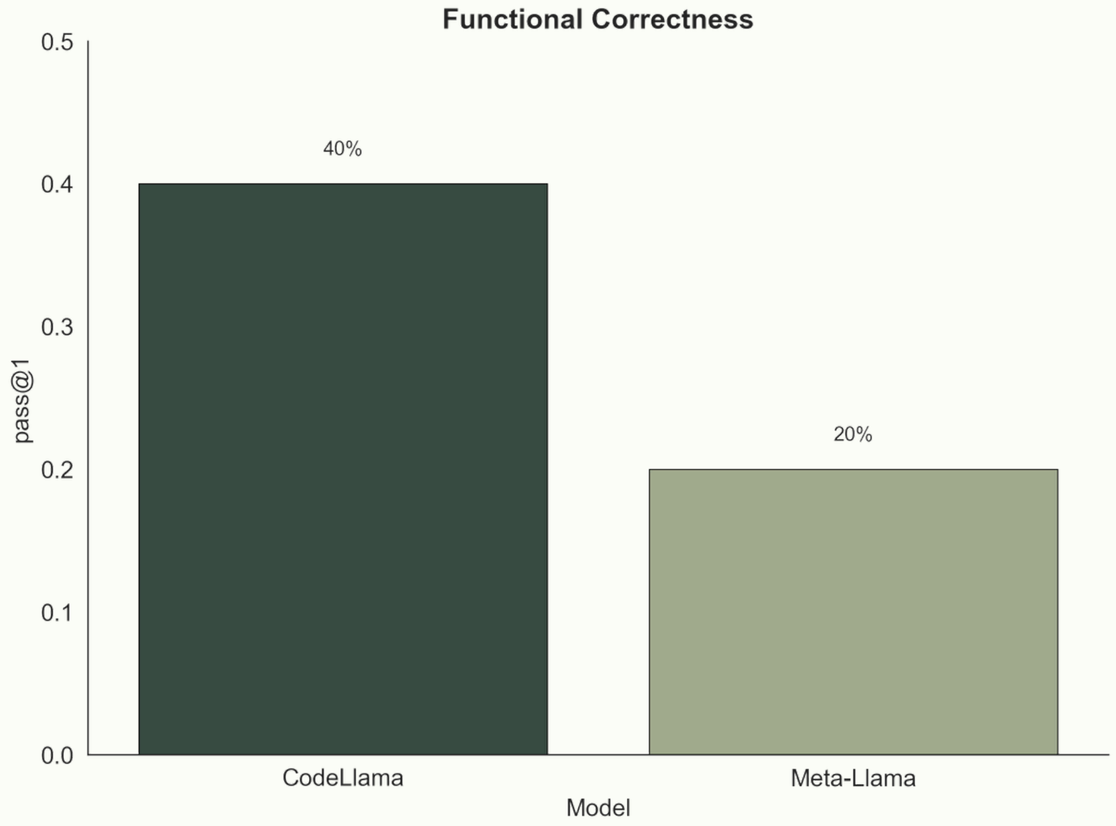


O CodeLlama apresenta um tempo de inferência médio de 18.1s enquanto o Meta-Llama demora em média 22.5s logo, se na escolha de um destes modelos se quisesse priorizar o tempo de inferência, a escolha seria o **CodeLlama**.

Nota: Para os valores dos gráficos foi usada a média entre os resultados obtidos com a estratégia 0-shot e com a estratégia 3-shot

7. c)

(c) Which model would you use for local Python code generation if you prioritize functional correctness, and why?

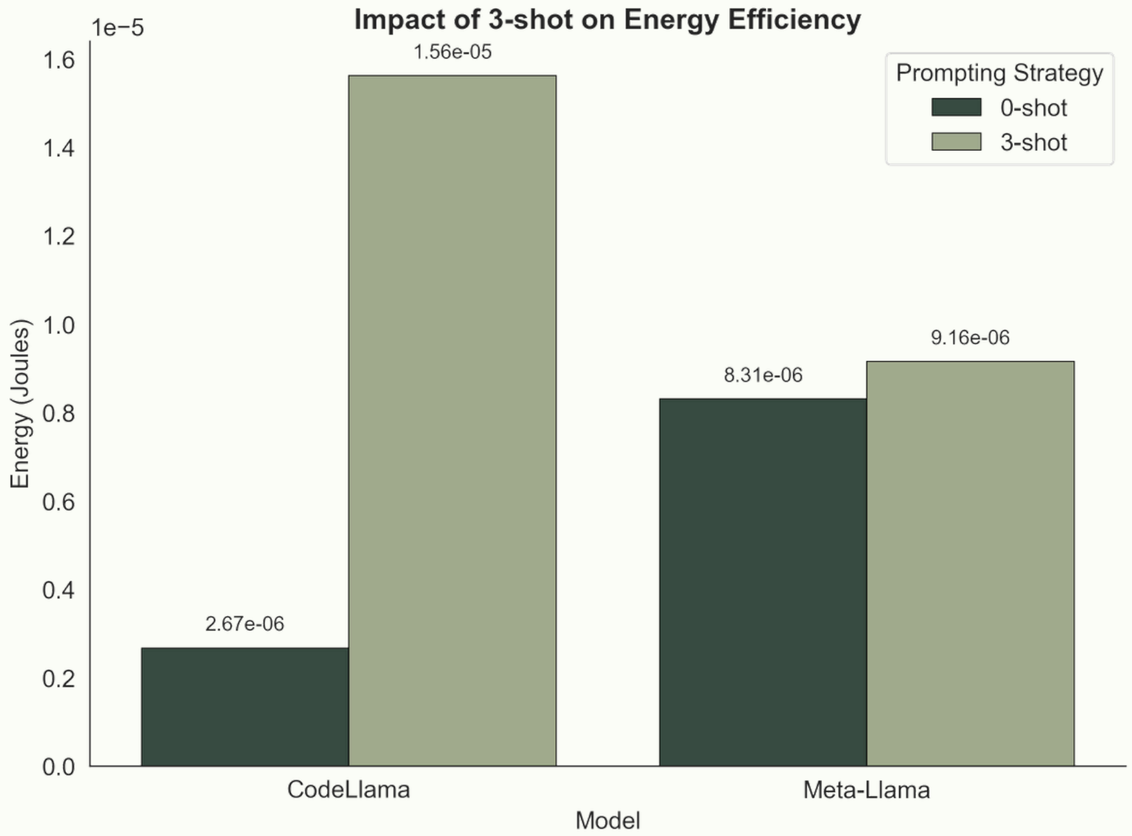


O CodeLlama apresenta uma média de 40% de taxa de sucesso no pass@1 enquanto que a taxa do Meta-Llama é de 20% logo, se na escolha de um destes modelos se quisesse priorizar a functional correctness, a escolha seria o **CodeLlama**.

Nota: Para os valores dos gráficos foi usada a média entre os resultados obtidos com a estratégia 0-shot e com a estratégia 3-shot

7. d)

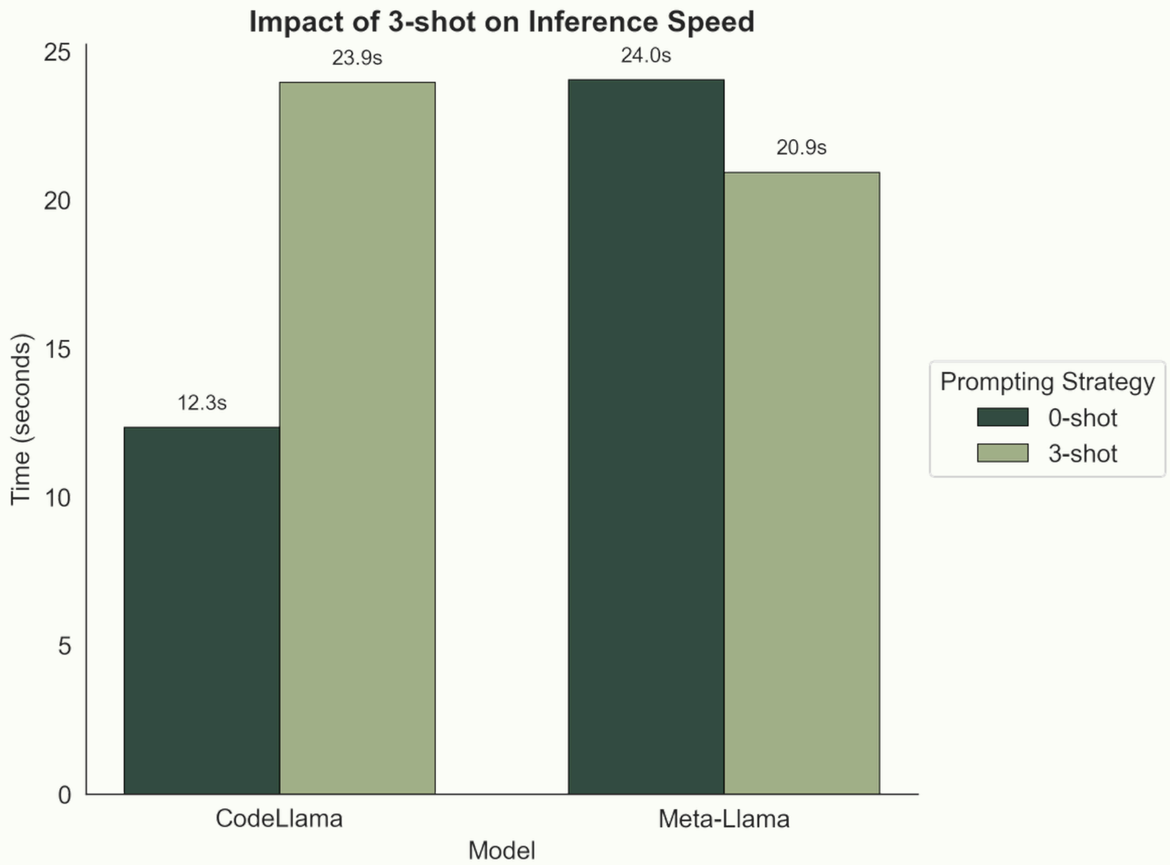
(d) How does the prompting strategy (0-shot vs 3-shot) influence the performance and energy consumption of each model?



O 3-shot aumenta o consumo energético em ambos os modelos, embora de forma mais acentuada no **CodeLlama**. Este modelo regista um aumento de aproximadamente **484%** no consumo energético quando se introduzem três exemplos. O **Meta-Llama**, por sua vez, apresenta um aumento mais modesto, de cerca de **10%.**

7. d)

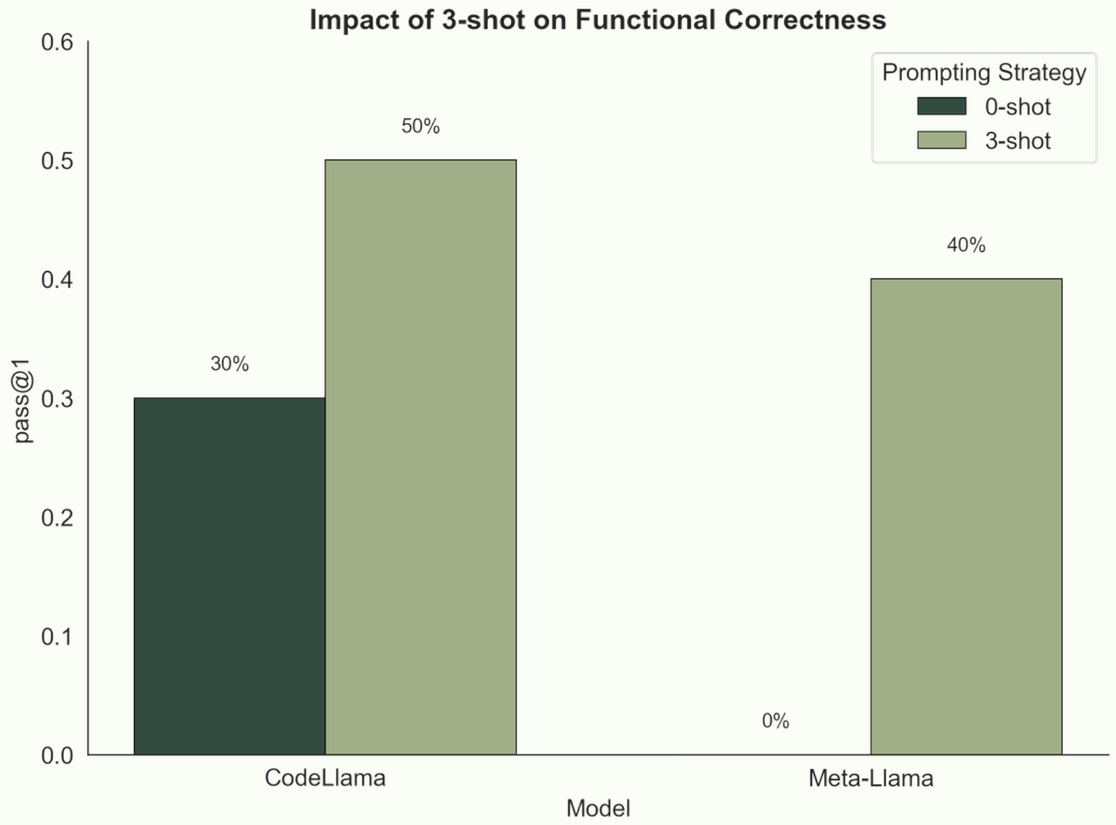
(d) How does the prompting strategy (0-shot vs 3-shot) influence the performance and energy consumption of each model?



O tempo de inferência também é afetado pela estratégia de prompting. O **CodeLlama** apresenta um aumento significativo no tempo de inferência, passando de 12.3s em 0-shot para 23.9s em 3-shot. O **Meta-Llama**, regista uma ligeira diminuição do tempo, de 24.0s para 20.9s

7. d)

(d) How does the prompting strategy (0-shot vs 3-shot) influence the performance and energy consumption of each model?



A estratégia 3-shot melhora a taxa de sucesso de ambos os modelos. O **CodeLlama** passa de 30% em 0-shot para 50% em 3-shot, enquanto o **Meta-Llama** apresenta uma melhoria ainda mais expressiva, de 0% para 40%. Este ganho é particularmente crítico para o Meta-Llama, que sem exemplos não consegue resolver qualquer tarefa corretamente.

8. pass@10 metric

	CodeLlama		Meta-Llama	
task	0-shot	3-shot	0-shot	3-shot
11	5/10	6/10	0/10	10/10
12	5/10	8/10	0/10	8/10
14	0/10	0/10	0/10	2/10
16	0/10	3/10	0/10	0/10
17	9/10	10/10	0/10	10/10
18	8/10	10/10	0/10	9/10
19	6/10	10/10	1/10	10/10
20	1/10	3/10	0/10	0/10
56	0/10	0/10	1/10	7/10
57	2/10	6/10	0/10	5/10

	CodeLlama		Meta-Llama	
task	0-shot	3-shot	0-shot	3-shot
58	1/10	10/10	1/10	9/10
59	0/10	0/10	0/10	0/10
61	0/10	2/10	2/10	3/10
62	8/10	10/10	0/10	10/10
63	0/10	0/10	0/10	3/10
64	8/10	9/10	0/10	6/10
65	4/10	10/10	1/10	7/10
66	9/10	10/10	0/10	10/10
67	0/10	0/10	0/10	0/10
68	4/10	4/10	0/10	0/10
total	13/20 65%	15/20 75%	5/20 25%	15/20 75%

8. pass@10 metric

CodeLlama:

- **0-shot:** obteve 65% de sucesso pois conseguiu resolver 13 tarefas de um total de 20, onde pelo menos um output cumpre todos os test cases
- **3-shot:** a adição de exemplos fez com que a taxa de sucesso aumentasse para 75% o que não parece um grande aumento mas ao verificar o número de outputs que passam nos testes verificamos que passa de 70 outputs para 111
- O caso mais extremo neste modelo é a tarefa 58 que ao usar a estratégia 0-shot tem apenas 1 output que passa em todos os testes, mas ao usar 3-shot, os 10 outputs gerados passam todos os testes

Meta-Llama:

- **0-shot:** obteve 25% de sucesso pois apenas conseguiu resolver 5 tarefas de um total de 20, onde pelo menos um output cumpre todos os test cases
- **3-shot:** com a adição de exemplos verificou-se um aumento significativo na taxa de sucesso, passando para 75%.

Conclusão: CodeLlama 3-shot e Meta-Llama 3-shot empatam com 75% de pass@10 mas se considerarmos o número de outputs válidos por tarefa, o CodeLlama 3-shot apresenta um maior número de sucesso tornando-se assim o melhor modelo a nível de functional correctness com 10 outputs